

This project is a *Blockchain-Based Identity Verification System*, and its main objective is to enhance the security and privacy when it comes to digital credentials. By using the **Blockchain** technology we assure that the data won't be changed (immutable), decentralized and hashed in the blockchain so it won't be revealed to anyone.

System Objective

The system will act as a verifier that checks if a specific user owns a specific credential, this way the system won't store any credentials or user's personal information, applying the decentralization mechanism to ensure that the customer has his own information not the system.

Main Actors

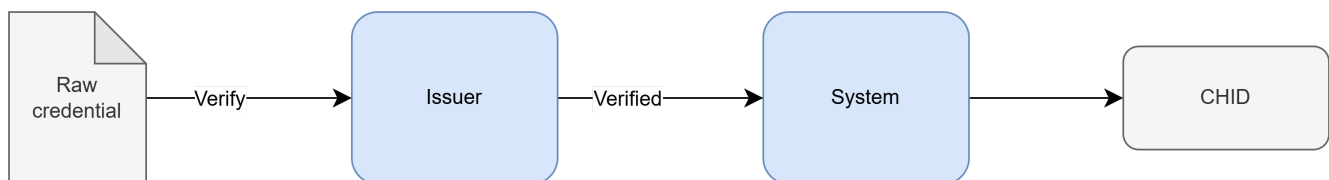
The system allows two types of primary actors to interact with it.

1. User (Holder):
2. Authority (Issuer)

And the blockchain network acts as a database API (Doesn't store credential), while the system is the verifier that decides if a specific credential belongs to a specific customer.

Data Flow

The initial step is to take the credential and verify it in the real-world-way (Issuer) that the user already the holder of it, once the issuer verifies the holder, the credential will be hashed and assigned to a **Credential Holder Identifier (CHID)**. The CHID is a hashed value that will be stored in the blockchain network referring to the credential holder and the credential itself.



Here is how the **Credential Holder Identifier (CHID)** looks like:

CHID = "CHID:HID:CID:AUTHID"

Where:

- **HID** → Holder ID (**User's ID**)

- **CID** → Credential ID
- **AUTHID** → Authority ID (**Authority ID**)

All of the IDs above are automatically generated by the system. However, both of the **user ID** and the **Authority ID** are the first IDs to be generated when registering to the system, and the **CHID** is generated when an authority approves that the credential belongs to the user.

System Paths

There are two main paths in this system, **Registering** and **Verifying**.

Registering

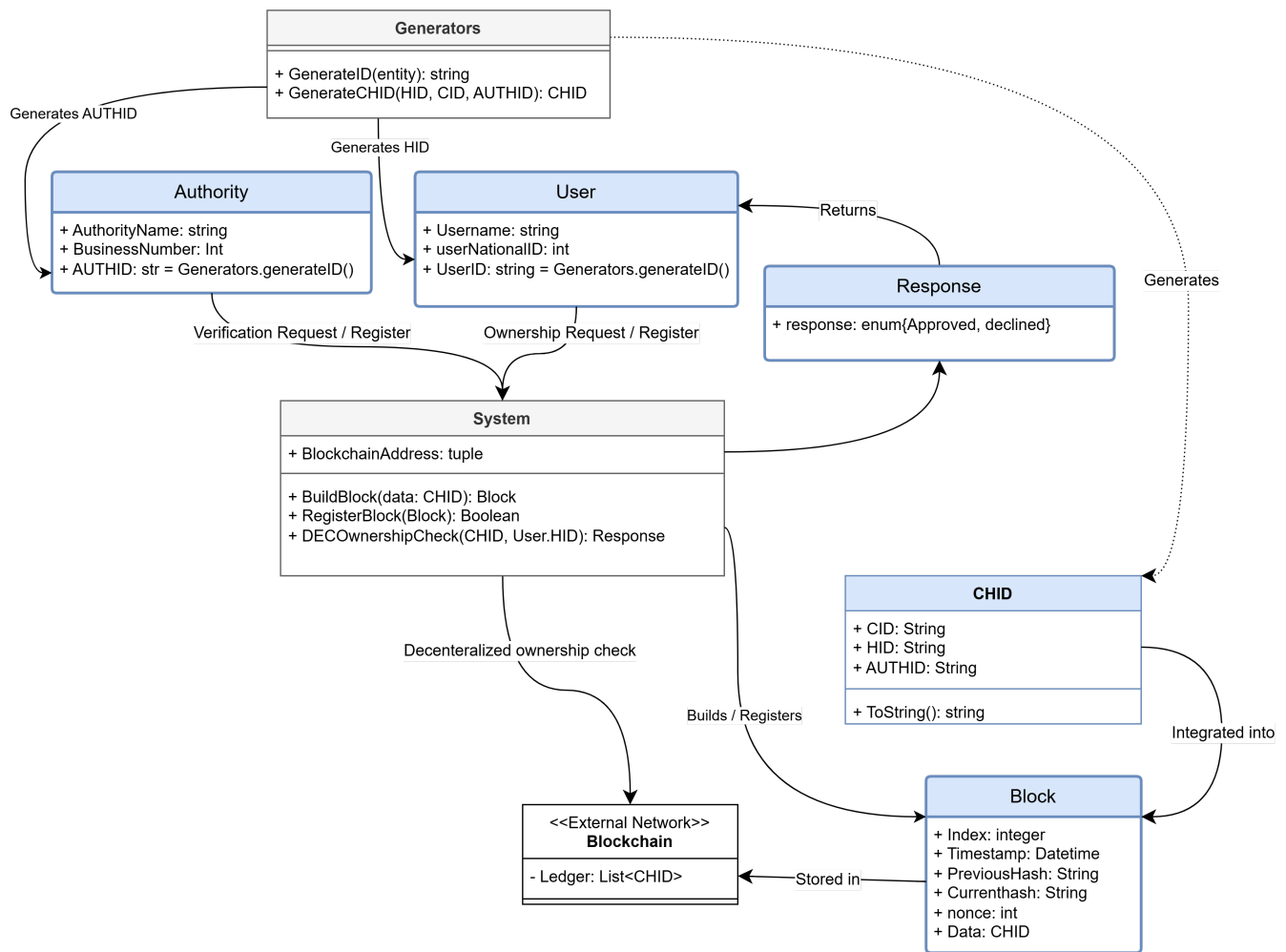
Each one of the actors will have it's own unique key (**User's Public Key**, **Authority Key**) wince they try to be registered in the system.

Also this is the step that involves the real-world approval by the the authority, if the authority approves that the user is the holder of the credential, it tells the system to generate a **CHID** and store it in the blockchain.

Verifying

Here is where the magic happens. After a specific credential is issued and hashed in the blockchain network, the system can check if the user owns a specific credential just by checking if the user's public key matches the **HID** inside the **CHID** address.

Class Diagram



How to Use

First initialize the user and the authority.

User:

```

user = User(
    username: str,
    nationalID: int,
    HID: str = Generators.generateID(str(User.__dict__)))
  
```

Authority:

```

authority = Authority(
    authorityName: str
    BusinessNumber: int
    AUTHID: str = Generators.generateID(str(Authority.__dict__))
)
  
```

After initializing the user and the authority, you can initialize the credential:

```
identity = Identity(  
    holder: User,  
    issuer: Authority,  
    image (Optional): str # Image path if exists.  
    CID: str = Generators.generateID(str(Identity.__dict__)),  
    CHID: CHID = Generators.generateCHID(holder, issuer, credentialID))
```

Registering to the Blockchain

```
# Creating the Block object with the data initialized above.  
block: Block = System.buildBlock(identity.CHID)  
  
# Register the block to the blockchain network  
System.registerBlock(block)
```

Verifying Credential

```
System.DECOwnershipCheck(identity: Identity, holder: User): Response
```

References